SQL AI Agent

Production-Grade Text-to-SQL System

with SQLAS Evaluation Framework

Case Study Submission

Author: Pradip Tivhale

Tech Stack: Python, FastAPI, React, Azure OpenAI, MLflow, SQLite

Package: sqlas v1.1.0 (published on PyPI)

Full-Stack

FastAPI + React
Azure OpenAI
MLflow Tracing

20 Metrics

SQLAS Framework
RAGAS-equivalent
Published on PyPI

Production

Read-only Safety
25 Test Cases
3-Layer Security

1. Executive Summary

This case study presents a production-grade SQL AI Agent system that translates natural language questions into SQL queries, executes them against any database, and returns human-readable answers. The system demonstrates advanced patterns for LLM-powered database agents including dynamic schema introspection, robust error handling with auto-retry, comprehensive MLflow observability, and a novel RAGAS-equivalent evaluation framework called SQLAS.

Key deliverables include: (1) A full-stack application with FastAPI backend and React frontend, (2) An MLflow-traced 3-stage agent pipeline, (3) A published Python package 'sqlas' on PyPI with 20 production-grade evaluation metrics across 8 categories, and (4) A 25-query test suite across 4 difficulty tiers with automated and LLM-based evaluation.

> *The SQLAS package (sqlas v1.1.0) is published on PyPI: pip install sqlas*

2. Problem Statement

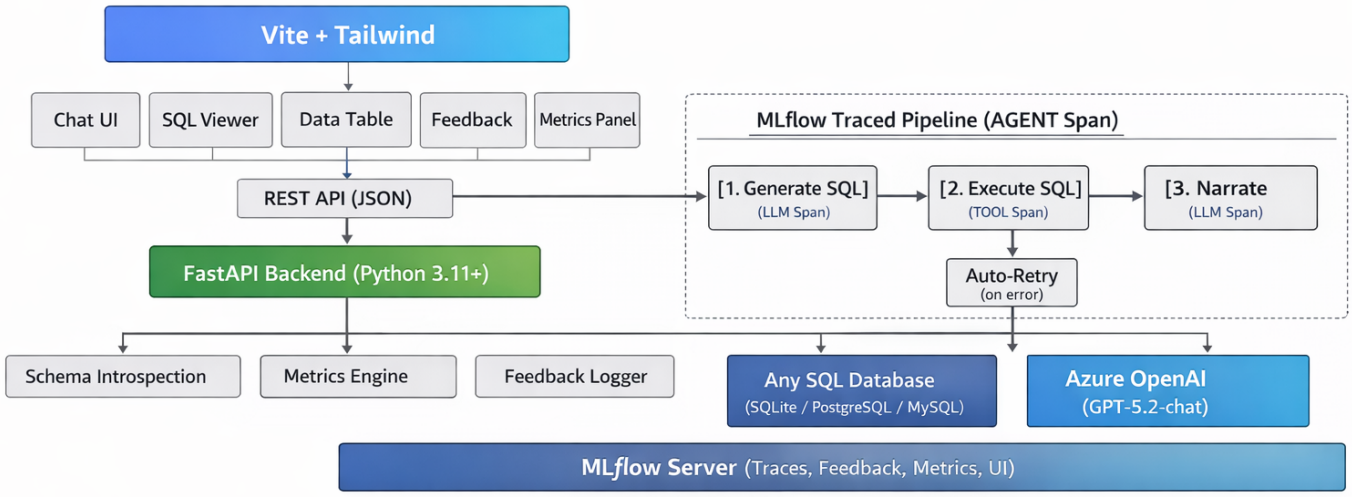
Healthcare organizations need to query patient databases to derive insights, but writing SQL requires specialized knowledge. The challenge is to build an AI agent that:

- Converts natural language questions to accurate SQL queries
- Handles complex analytics (JOINs, aggregations, correlations, CASE WHEN)
- Ensures strict read-only safety (no data modification)
- Provides transparent, traceable results with full observability
- Can be evaluated systematically with production-grade metrics

The dataset contains 2,000 patients with 14 health/demographic variables and 20,000 daily physical activity records (10 days per patient), stored in two related SQLite tables.

3. Solution Architecture

3.1 System Overview



3.2 Tech Stack

Layer	Technology	Purpose
Backend	Python 3.11 + FastAPI	Async REST API, agent orchestration
LLM	Azure OpenAI (GPT-5.2-chat)	SQL generation + narration
Database	SQLAlchemy 2.0 + aiosqlite	Async DB access, schema introspection
Observability	MLflow 3.10.1	Tracing, spans, feedback, metrics
Evaluation	SQLAS (sqlglot)	20-metric evaluation framework
Frontend	React 18 + Vite + Tailwind	Chat UI, SQL viewer, data table
Data	SQLite (2 tables, 22K rows)	Health demographics + physical activity

3.3 Project Structure

File	Lines	Purpose
backend/agent.py	175	3-stage agent pipeline (generate, execute, narrate)
backend/tracing.py	299	MLflow tracing with hierarchical spans
backend/database.py	266	Dynamic schema introspection, read-only execution
backend/eval_framework.py	988	SQLAS: 15 metrics, 6 categories
backend/eval_runner.py	538	25 test cases, 4 difficulty tiers
backend/main.py	144	FastAPI routes (query, feedback, schema, eval)
frontend/src/ (6 files)	700+	React chat UI with metrics, feedback, SQL viewer
sqlas-package/	1200+	Published PyPI package (sqlas v1.1.0)

4. Agent Pipeline (3 Stages)

4.1 Stage 1: SQL Generation

The LLM receives the user's question along with dynamically introspected database context (tables, columns, types, statistics, sample rows, foreign keys). A 64-rule system prompt guides SQL generation including CTE usage, NULL handling, CASE WHEN for labels, and aggregation strategy (aggregate N-side first to prevent row explosion).

4.2 Stage 2: SQL Execution with Auto-Retry

SQL is executed against the database through a read-only enforced connection. If execution fails, the error message is fed back to the LLM which generates a corrected query. Both attempts are logged as separate MLflow spans.

4.3 Stage 3: Result Narration

The LLM converts raw query results into a plain English answer. Rules enforce: direct answer first, no markdown tables for single values, no health disclaimers. SQL, data, and metrics are shown in expandable UI panels.

4.4 Example Flow

```
User: "How many patients have abnormal blood pressure?"

SQL Generated:
  SELECT COUNT(*) FROM health_demographics
  WHERE Blood_Pressure_Abnormality = 1

Executed: 1.44ms, 1 row returned

Narrated: "987 patients have abnormal blood pressure."

MLflow Trace: generation=3.4s, execution=1.4ms, narration=2.9s
```

5. Key Features

5.1 Dynamic Schema Introspection

On startup, the system auto-discovers any database schema without hardcoded knowledge. It extracts table names, column types, primary/foreign keys, indexes, column statistics (min/max/avg for numeric, top values for categorical), and sample rows. This context is injected into the LLM prompt, enabling accurate SQL generation for any database.

5.2 3-Layer Read-Only Safety

Layer	Mechanism	Catches
1. LLM Prompt	System message forbids DDL/DML	Most cases (LLM follows instructions)
2. SQL Validation	Pre-execution keyword check	INSERT, UPDATE, DELETE, DROP, etc.
3. Keyword Blocking	14 forbidden SQL keywords	TRUNCATE, GRANT, REVOKE, ATTACH...

5.3 MLflow Observability

Every query creates an MLflow trace with hierarchical spans:

```
sql_agent_pipeline (AGENT)
  |-- generate_sql (LLM)     -> latency, sql_metrics, token_usage
  |-- execute_sql (TOOL)     -> execution_time, row_count
  |   |-- [retry_generate]   -> corrected SQL (if error)
  |   +-- [retry_execute]    -> retry result
  +-- narrate_result (LLM)   -> response_length, narration_latency
```

16+ metrics captured per query: latencies (generation, execution, narration), SQL complexity (joins, CTEs, subqueries, aggregation), result stats (rows, columns), and retry count.

5.4 User Feedback Collection

The UI provides thumbs up/down feedback with optional comments, plus a detailed rating system (accuracy, relevance, SQL quality on 1-5 scale). All feedback is logged to MLflow traces for quality tracking and model improvement.

5.5 On-the-Fly JOINs

Datasets remain as separate tables. The LLM generates JOINs only when cross-table analysis is needed. Smart aggregation strategy (aggregate N-side first) prevents row explosion. Example: 'Average steps by CKD status' generates a JOIN with GROUP BY on the demographics side.

6. SQLAS Evaluation Framework

> *SQLAS (SQL Agent Scoring) is a RAGAS-equivalent evaluation framework for SQL AI agents. Published as an open-source Python package on PyPI (pip install sqlas). 20 metrics across 8 categories. LLM-agnostic. Production-hardened.*

6.1 Why SQLAS?

Existing evaluation frameworks like RAGAS are designed for RAG pipelines, not SQL agents. SQL agents need different metrics: execution correctness (does the SQL return right data?), schema awareness (are the right tables/columns used?), query efficiency (is it optimized?), and safety (no destructive operations or PII exposure). SQLAS fills this gap.

6.2 20 Metrics Across 8 Categories

Category	Metric	Weight	Type
Execution Accuracy (35%)	execution_accuracy	35%	Automated
Semantic Correctness (13%)	semantic_equivalence	13%	LLM Judge
Context Quality (10%)	context_precision	3%	Automated
	context_recall	3%	Automated
	entity_recall	2%	Automated
	noise_robustness	2%	Automated
Cost Efficiency (12%)	efficiency_score	4%	Automated
	data_scan_efficiency	4%	Automated
	sql_quality	2%	LLM Judge
	schema_compliance	2%	Automated
Execution Quality (8%)	execution_success	4%	Automated

Category	Metric	Weight	Type
	complexity_match	2%	LLM Judge
	empty_result_penalty	2%	Automated
Task Success (8%)	faithfulness	3%	LLM Judge
	answer_relevance	2%	LLM Judge
	answer_completeness	2%	LLM Judge
	fluency	1%	LLM Judge
Result Similarity (4%)	result_set_similarity	4%	Automated
Safety (10%)	read_only_compliance	5%	Automated
	safety_score	5%	Automated

6.3 RAGAS to SQLAS Mapping

RAGAS Metric	SQLAS Equivalent	SQL-Domain Meaning
Faithfulness	faithfulness	Claims grounded in SQL result data
Answer Relevance	answer_relevance	Response answers the question
Answer Correctness	execution_accuracy	SQL returns correct results
Answer Similarity	result_set_similarity	Result set Jaccard similarity
Context Precision	context_precision	Only relevant schema elements used
Context Recall	context_recall	All required schema elements used
Context Entity Recall	entity_recall	Strict entity match (tables, cols, literals)
Noise Sensitivity	noise_robustness	Resistance to irrelevant schema

6.4 Novel Execution Accuracy Formula

Unlike Spider/BIRD (exact-match), SQLAS uses semantic comparison:

```
Execution Accuracy = 60% x Output Match
                   + 20% x Structure Match
                   + 20% x Efficiency Score

Output Match:    Row-by-row numeric comparison with tolerance.
                 Ignores label differences (0 vs 'Male'),
                 tolerates ROUND variations.
Structure Match: Same row count? Both return data?
Efficiency:      min(gold_time / pred_time, 1.0)
```

6.5 Production Hardening

- Read-only database connections (sqlite3 ?mode=ro)
- 30-second SQL execution timeout guard
- LLM judge error handling (all 7 LLM metrics wrapped in try/except)
- Input validation (empty SQL, missing db_path, weight sum checks)
- PII detection with word-boundary matching (no false positives)
- Structured logging via Python logging module
- py.typed marker for mypy/pyright type checking

7. Test Suite

7.1 25 Test Cases x 4 Difficulty Tiers

Tier	Count	Complexity	Example Question
Easy	7	Single table, simple filter	How many patients have abnormal BP?
Medium	8	GROUP BY, multi-condition	Avg BMI for smokers vs non-smokers?
Hard	7	Cross-table JOINs	Avg daily steps by CKD status?
Extra Hard	3	CTEs, correlations	Pearson correlation: hemoglobin vs activity?

7.2 Verification Results (111/111 Tests Passed)

The SQLAS PyPI package was comprehensively tested against the SQL AI Agent project using the real health.db database. All 111 test assertions passed covering:

Test Category	Tests	Status
Imports & Version	7	PASS
Database Connectivity	5	PASS
Execution Accuracy (easy/medium/hard)	5	PASS
Syntax Validity	5	PASS
Schema Compliance	4	PASS
Context Precision (RAGAS)	4	PASS
Context Recall (RAGAS)	3	PASS
Entity Recall (RAGAS)	4	PASS
Noise Robustness (RAGAS)	3	PASS
Result Set Similarity (RAGAS)	5	PASS
Data Scan Efficiency	3	PASS
Safety & Governance	9	PASS
LLM-Based Metrics (mock judge)	8	PASS
LLM Error Resilience	7	PASS
Full Evaluate Pipeline	17	PASS
WEIGHTS_V2 Scoring	2	PASS
Evaluate Without Gold SQL	3	PASS
Input Validation	2	PASS
Batch Evaluation	4	PASS
Composite Scoring (v1 + v2)	3	PASS
All 25 Gold SQL Test Cases	8	PASS

8. Frontend UI

Built with React 18 + Vite + Tailwind CSS. Dark theme with indigo accent. Features include:

- Chat interface with auto-scroll and loading indicators
- Expandable SQL viewer with syntax highlighting and copy button
- Scrollable data table with CSV export and row count

- Thumbs up/down feedback with optional comment box
- Metrics panel showing 12+ metrics in a 2x6 grid
- SQL feature tags (Aggregation, GROUP BY, CASE WHEN, etc.)
- Sidebar with database status, schema explorer, and sample queries
- Conversation management with clear history

8.1 Component Architecture

```
App.jsx (root state, API calls)
  +-- Sidebar.jsx (DB status, schema explorer, sample queries)
  +-- ChatInterface.jsx (message list, input bar)
       +-- MessageBubble.jsx (response, feedback, metrics)
            +-- CodeBlock.jsx (expandable SQL viewer)
            +-- DataTable.jsx (scrollable result table)
```

9. API Endpoints

Method	Endpoint	Description
GET	/health	Database & MLflow status
GET	/schema	Auto-discovered schema with statistics
POST	/query	Main query endpoint (NL -> SQL -> Execute -> Narrate)
POST	/feedback	Thumbs up/down feedback
POST	/feedback/detailed	Multi-dimension rating (accuracy, relevance, quality)
POST	/evaluate	Run SQLAS evaluation suite (quick=5, full=25 tests)
DELETE	/conversations/{id}	Clear conversation history

10. Key Design Decisions

Decision	Choice	Rationale
Data Processing	SQL (not Pandas)	Scales to billions of rows, indexed queries, safe
Schema Discovery	Dynamic introspection	Works with any DB, no hardcoded knowledge
Table Strategy	On-the-fly JOINs	Avoids data merge complexity, aggregate-first
Safety Model	3-layer enforcement	Defense in depth: prompt + validation + blocking
Error Handling	Auto-retry with LLM	Most SQL errors fixable, improves success rate
Observability	MLflow tracing	Hierarchical spans, token usage, feedback loops
Evaluation	SQLAS (semantic)	SQL-specific metrics, not exact-match like Spider
LLM Interface	LLM-agnostic	Works with OpenAI, Anthropic, local models

11. Deliverables Summary

Deliverable	Details
Full-Stack Application	FastAPI backend + React frontend, fully functional
SQL AI Agent Pipeline	3-stage: generate, execute, narrate with auto-retry
MLflow Observability	Traces, spans, 16+ metrics, feedback collection
SQLAS Package (PyPI)	sqlas v1.1.0 - 20 metrics, 8 categories, published
Test Suite	25 queries x 4 tiers, 111 test assertions, all passing
Documentation	DOCUMENTATION.md (506 lines), PDF docs, CHANGELOG
Safety System	3-layer read-only, PII detection, injection prevention
Dynamic Schema	Auto-discovers any database, statistics-enriched prompts

All code is available for review. The SQLAS package is live on PyPI: pip install sqlas